

# Software Requirements Specification (SRS)

---

## SoundScript – Client-Side Music Streaming Platform

Prepared by: Dipsana Roy

---

*Version: v1.0.2*

---

---

## 1. Introduction

### 1.1 Purpose

SoundScript is a lightweight, frontend-only music streaming application built with modular Vanilla JavaScript. It provides a responsive, event-driven music playback experience without requiring a backend server or user accounts. The application is designed to demonstrate scalable frontend architecture, efficient state management, and clean modular software engineering while maintaining a minimal deployment footprint.

---

### 1.2 Scope

SoundScript enables users to browse albums, play music, manage playback, and maintain personal listening statistics entirely within the browser.

The application:

- Streams media from configurable static assets (GitHub CDN or local assets)
- Automatically discovers albums without hardcoded configuration
- Maintains playback statistics locally using LocalStorage
- Provides responsive layouts across desktop and mobile devices
- Uses an event-driven playback engine for synchronized UI updates
- Operates without authentication, databases, or backend services

The platform is designed as a **logic plane** separated from its **media plane** using a dedicated CDN repository.

---

### 1.3 Definitions

| Term | Description                                        |
|------|----------------------------------------------------|
| CDN  | External static media repository (soundscript-cdn) |

|               |                                                   |
|---------------|---------------------------------------------------|
| Queue         | Ordered track list currently loaded               |
| Media Plane   | CDN asset layer (audio, images, metadata)         |
| Logic Plane   | SoundScript app layer (HTML, CSS, JS)             |
| Mini Bar      | Draggable compact playback control UI             |
| Seek Bar      | Audio timeline controller with gradient sync      |
| Event Emitter | Custom publish-subscribe system for player events |

---

## 2. Overall Description

### 2.1 Architecture Overview

SoundScript follows a modular frontend architecture where each module has a single responsibility and communicates through controlled interfaces and custom events. Critical application state is privately managed and exposed only through getter functions, preventing accidental mutation by other modules.

---

### 2.2 Product Perspective

SoundScript is a standalone single-page application (SPA) built entirely with HTML, CSS, and Vanilla JavaScript. Media assets are delivered through configurable static sources, allowing deployment either with local assets or a static CDN.

No backend infrastructure is required.

---

### 2.3 Product Functions

- Dynamic album discovery
  - Event-driven playback engine
  - Queue management
  - Responsive player interface
  - Floating mini player (Precise 2D mini-bar drag)
  - Playback statistics
  - LocalStorage persistence
  - Partial title search
  - Touch-enabled card sliders
  - Automatic UI synchronization
  - Configurable media source
  - SPA navigation
  - Track plays using heuristic logic ( $\geq 2$  min or  $\geq 80\%$  listened)
  - Animate and adapt UI dynamically (gradient shifts, transitions)
  - Instant section loading
-

## 2.4 User Classes

The application targets users seeking a lightweight local music player as well as developers interested in studying modular frontend architecture and event-driven UI design.

---

## 2.5 Constraints

- Frontend-only architecture
  - No backend dependencies
  - Browser LocalStorage availability
  - JavaScript enabled
  - Static hosting compatibility
- 

## 2.6 Assumptions

- Browser supports ES6 Modules
  - Browser supports ResizeObserver
  - Browser supports LocalStorage
  - Browser supports CSS Grid and Flexbox
  - Audio playback APIs are available
- 

## 2.7 Operating Environment

| Component | Description                                                                              |
|-----------|------------------------------------------------------------------------------------------|
| Browser   | Modern Chromium / Gecko (Chrome, Firefox, Edge)                                          |
| Hosting   | Localhost, GitHub Pages, any static host                                                 |
| CDN       | GitHub Raw ( <a href="https://raw.githubusercontent.com">raw.githubusercontent.com</a> ) |

---

# 3. System Features

## 3.1 Functional Requirements

### *FR-1 Album Discovery*

The system shall automatically discover available albums without requiring hardcoded album registration.

### *FR-2 Music Playback*

The system shall provide:

- Play
- Pause
- Next
- Previous
- Seek
- Repeat
- Queue management

#### *FR-3 Playback Statistics*

The system shall maintain:

- Play count
- Likes
- Dislikes
- Trending songs
- Loved songs
- Hated songs

using LocalStorage.

#### *FR-4 Navigation*

The system shall:

- Navigate without page reloads
- Synchronize playback state
- Preserve player state across navigation

#### *FR-5 Responsive Interface*

The interface shall automatically adapt to:

- Desktop
- Laptop
- Tablet
- Mobile

#### *FR-6 Search*

The application shall support partial title searching with automatic result focusing.

---

## **3.2 Album Discovery Module**

The Album Discovery Module dynamically detects music albums and generates their user interface without requiring manual registration.

## Capabilities

- Automatically discovers albums from the configured media directory
  - Loads album metadata from info.js
  - Dynamically generates album and song cards
  - Binds playback events to generated tracks
- 

## 3.3 Playback Engine

The Playback Engine controls all audio playback operations through a modular event-driven architecture.

### Capabilities

- Event-driven playback controller
  - Queue management
  - Play, pause, next, previous and repeat controls
  - Playback state synchronization
  - O(1) runtime access after initialization
  - Volume and playback persistence
- 

## 3.4 Seek Bar System

The Seek Bar provides synchronized playback navigation.

### Capabilities

- Real-time playback synchronization
  - Desktop drag seeking
  - Mobile touch seeking
  - Gradient synchronization with the background engine
  - Continuous playback position updates
- 

## 3.5 Responsive User Interface

The interface adapts automatically to different screen sizes while maintaining consistent interaction.

### Capabilities

- Mobile-first responsive layout
- CSS-only hamburger navigation
- Floating draggable mini player

- Touch-enabled card sliders
  - Smooth layout transitions
- 

### **3.6 Playback Statistics Engine**

The Statistics Engine maintains persistent playback information locally.

#### **Capabilities**

- Play count tracking
  - Like and dislike tracking
  - Trending, loved and hated song calculation
  - LocalStorage persistence
  - No external data transmission
- 

### **3.7 Navigation & Loading System**

The Navigation System provides SPA-like transitions without page reloads.

#### **Capabilities**

- Instant section switching
  - Manual loading controller
  - Flash-free page transitions
  - Persistent playback during navigation
  - Lazy generation of additional content
- 

## **4. Non-Functional Requirements**

### **Performance**

- Initial UI loads in under 2 seconds on modern broadband.
  - Album generation is lazy-loaded.
  - Playback transitions occur without perceptible delay.
- 

### **Reliability**

- Playback statistics survive browser restarts.
- Invalid slider calculations are safely ignored.
- UI state remains synchronized during navigation.

---

## Maintainability

- Modular JavaScript architecture
  - Private state encapsulation
  - Event-driven communication
  - Consistent naming conventions
- 

## Portability

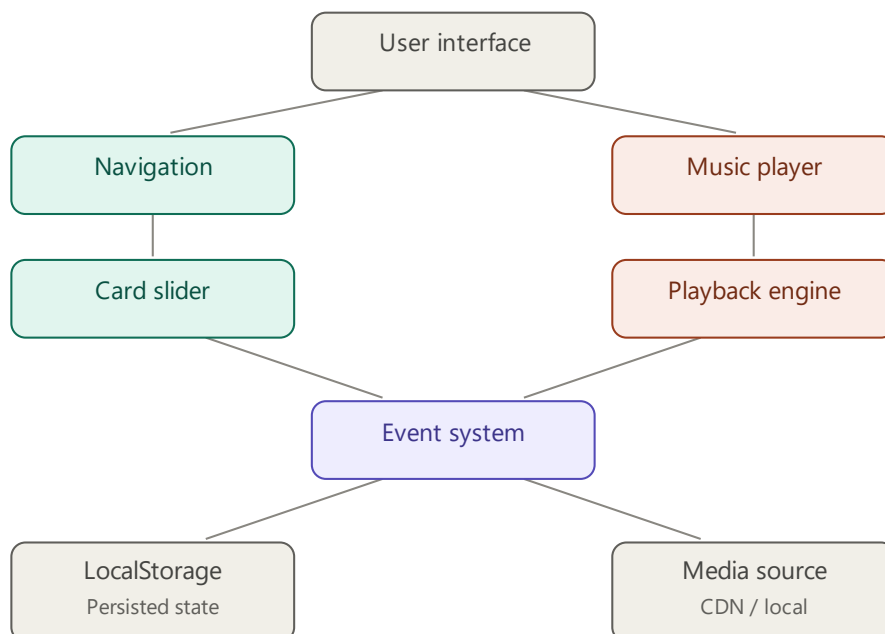
Runs on

- Chrome
- Firefox
- Edge
- Brave
- Opera

without additional installation.

---

## 5. System Architecture



---

## 6. Data Storage

| Data                | Storage       |
|---------------------|---------------|
| Playback Statistics | LocalStorage  |
| Queue State         | Memory        |
| Player State        | Memory        |
| Albums              | JSON          |
| Media               | Static Assets |

---

## 7. Security

SoundScript stores only non-sensitive playback information locally. No personally identifiable information is collected or transmitted. Because the application has no backend services or authentication system, its attack surface remains intentionally minimal.

---

## 8. Future Enhancements

- Custom playlist creation
  - Shuffle playback
  - Enhanced search and filtering
  - Automatic local music library discovery
  - Automatic metadata generation
  - Album artwork generation
  - Mobile UI refinements
  - SVG icon system
- 

## 9. Version History

| Version | Date       | Highlights                                                                                |
|---------|------------|-------------------------------------------------------------------------------------------|
| v1.0.2  | 2026-08-04 | Synchronization, responsive fixes, configurable media delivery, improved slider stability |
| v1.0.1  | 2026-01-05 | Mobile drag fixes, swipe logic, loader refactor, UI polish, bug squashing                 |
| v1.0.0  | 2026-01-04 | Initial stable production release with full CDN architecture                              |

---

## 10. Conclusion

SoundScript demonstrates that a modern music streaming experience can be achieved using a fully client-side architecture without backend infrastructure or third-party frameworks.

Through modular Vanilla JavaScript, dynamic album discovery, event-driven playback, and persistent local storage, the application provides a responsive and maintainable platform while preserving user privacy by keeping all playback data on the user's device. Future versions will continue to enhance usability, accessibility, and local media management while maintaining the project's lightweight, frontend-only design.